

HiSPARC Cosmic Ray Detector Experiment

Data Analysis with Python SAPPHiRE Package

Christoffel Schoeman
Student Number: 38655349
NWU

October 7, 2025

Contents

1	Abstract	3
2	Introduction	3
3	Method	3
4	Results	5
5	Pulseheight Histograms and MPV Values	5
6	Temperature Data	6
7	Coincidence Analysis	6
8	Direction Reconstruction	7
9	Discussion	7
10	Conclusion	8
11	AI Usage	10

1 Abstract

2 Introduction

HiSPARK, which stands for the "High School Project on Astrophysics Research with Cosmics" is a project set up by several different high schools and academic institutions to observe and measure cosmic rays[1].

Cosmic rays are interstellar, high energy particles that enter the Earth's atmosphere. Upon entering the Earth's atmosphere, the particles collide with atmospheric particles. From these collisions, the atmospheric particles become heavily energised and cascade downwards, hitting other particles and energising them.

HiSPARK is a ground based array, which are objects like plastic scintillators, fluorescence telescope, and so on[2]. Each of these measure cosmic rays in unique methods, but largely achieve the same effect. Within HiSPARK is a number of stations one can select from, and as well as select date ranges from for further cosmic ray calculations. This is explored further within the method.

Within this report is the analyses of several characteristics of cosmic rays. Namely:

- Pulseheight distributions
- Most Probable Values (MPVs)
- Coincidence Events
- Weather condition factors (such as temperature and pressure)

The main focus here is to document these measurements and extract conclusions from them.

3 Method

The bulk of this method will focus primarily on python code and the use of the several packages. The python script used for this report contains the following packages:

- Sapphire (for accessing station data)
- Tables
- datetime
- Numpy (for assistance in calculations)
- Matplotlib (for figures)

The structure of the program is as follows:

1. Import all of the mentioned libraries above for full functionality.
2. Open a data file called "datafile", in this case.
3. Declare start and end time variables. In this example, the date 19 August 2018's data was used.
4. Download data into the datafile
5. Extract events from the datafile
6. Extract the "pulseheights" data from the pulseheights column under events
7. Plot a histogram, using matplotlib.pyplots.
8. Download the weather data into the datafile (this falls under sapphire functionality)
9. Extract pressure, time, temperature inside, timestamps and temperature outside within the columns of the weather dataset.
10. Similar to the pulseheights, plot line graphs of temperature against time.
11. Using Sapphire analysis, specifically the functionality to find most probably values, extract the MVP for each of the detectors. To do this, extract bin number and number of pulseheights from the histogram.
12. Import the two derived values into the FindMostProbablyValueInSpectrum function.
13. Use this variable and apply the $find_{mpv}()$ function to return the most probably value for that detector.
14. repeat this process 3 more times for the other 3 detectors
15. Plot histograms of the pulseheights of each of the detectors with their MVP marked on the diagram, again making use of matplotlib.
16. For the coincidences section, close the previous file and open a new file. This is because a weeks worth of data will now be used. For this case, the week of 13 August 2018 to 19 August 2018 was used.
17. Open a new file called coinTable.
18. Download data from 4 different sites. For this report, the stations 501, 504 and 509 were used.
19. Download events from these stations.
20. Use the CoincidencesESD function to get critical information for coincidences calculations.
21. Make use of the search and store coincidences function

22. Using this data, calculate total coincidences found, number between two stations, number between three stations and the percentage of events that were coincidences.
23. For the last section, make use of the `ReconstructESDEvents` to estimate cosmic ray directions.
24. Use the function on the `coinData` set.
25. From this, zenith and azimuth values can be extracted.
26. Convert these zenith and azimuth values to degrees from radians
27. Create a plot, using `matplotlib`, of a histogram of the zenith angles
28. Create a polar plot (scatter plot) of both the azimuth and zenith angles.

4 Results

5 Pulseheight Histograms and MPV Values

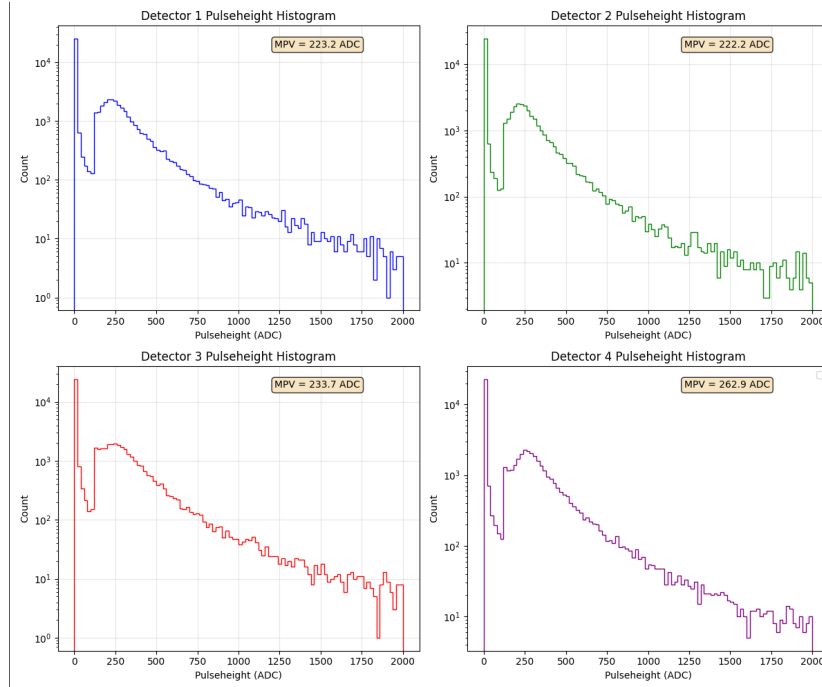


Figure 1: Pulseheight histograms for all four detectors at station 501.

The Most Probable Values (MPV) for each detector are:

- Detector 1: 223.2 ADC
- Detector 2: 222.2 ADC
- Detector 3: 233.7 ADC
- Detector 4: 262.9 ADC

6 Temperature Data

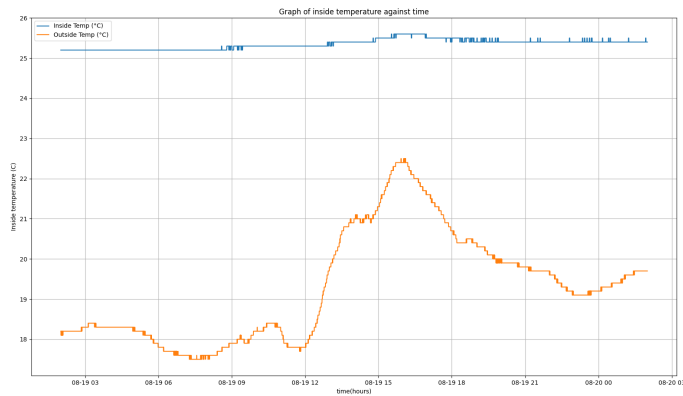


Figure 2: Inside and outside temperature versus time.

Temperature units: Celsius ($^{\circ}\text{C}$), based on typical room temperature values.

7 Coincidence Analysis

Stations: 501, 504, 509

Date range: 2018-08-13 to 2018-08-19 (7 days)

Total coincidences: 2606

Two station: 2393

Three station: 213 Coincidence percentage: 0.2608%

8 Direction Reconstruction

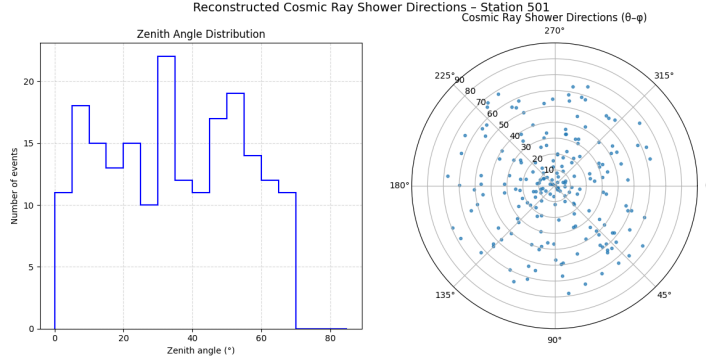


Figure 3: Polar plot showing azimuth and zenith angles of cosmic ray arrival directions and zenith histogram.

9 Discussion

At first looking at the pulseheights graph, it is clear to see that bulk of the detectors follow a similar structure in their detection histograms. Their MPVs are all around the same value of 225 ADC give or take, except for detector 4 which has a high MPV value of 262.9ADC. It could be that perhaps this detector is at a higher altitude and got more intense cosmic radiation landing on it.

For the temperature values, the values seem to "hover" around average room temperature values. Although one should note that internal temperatures are on average higher than that of the external temperature. Regarding the internal temperature it seems to peak at around 15:00, which seems to make sense as that is when the sun shines brightest on the device assume that it is exposed to sunlight.

For coincidence calculations, a very small amount of events seem to be coincidences. This implies that a very small amount of all recorded incidents are from the same cosmic ray event, or that perhaps the cosmic ray event was not especially powerful. Perhaps a repeat on different days could yield higher percentage values.

For direction reconstruction, looking at the histogram most arrive at angles 0 to 55 degree angels, with a large peak at around 30 degrees. Not a lot of cosmic rays arrived at steeper angles higher than 70 degrees. For the polar plot each dot represents a cosmic ray entering in that region. Ones arriving in the centre arrived on overhead of the detector. The distribution appears to be rather uniform.

10 Conclusion

The aim of this report was to explore the capabilities of the sapphire library under HiSPARK context and to explore the nature of cosmic rays more deeply. While some improvements can be made, and repetition of this report under different stations and at different dates could yield provide further understanding of sapphire as a whole, the program is an incredibly powerful and potent library.

Not only is it useful as an educational library for both high school and university students, but also in other contexts such as the aviation industry as key data can be extrapolated from this library. For aviation specifically, cosmic rays are a big concern of theirs for their pilots and flight attendants that fly often.

Overall, the library is incredibly potent and useful. It can be made more useful with more up to date information and can perhaps pair nicely with other programs such as MongoDB that is connected to some cosmic ray detector for efficient cosmic ray interpretations.

References

- [1] HiSPARC Utah. Hisparc - the university of utah, 2025. Accessed: 2025-10-07.
- [2] Pravata Kumar Mohanty. Cosmic ray sources and detectors. *The European Physical Journal Special Topics*, 234(6):1–21, 2025. Accessed: 2025-10-07.

11 AI Usage

AI such as Claude, Chatgpt and Grok were used extensively for the code and assistance in writing the code and debugging it.